Turbos Clmm

Audit Report

contact@bitslab.xyz

https://twitter.com/movebit_

Tue Dec 17 2024

Turbos Clmm Audit Report

1 Executive Summary

1.1 Project Information

Description	DEX
Type	Dex
Auditors	MoveBit
Timeline	Thu Dec 05 2024 - Tue Dec 17 2024
Languages	Move
Platform	Sui
Methods	Architecture Review, Unit Testing, Manual Review
Source Code	https://github.com/turbos-finance/turbos-clmm
Commits	487e891e4cdc4450a0cb172ed4bd01a2692320df b657a46a3cd4cc77091abedd94e73da4d11c67b2

1.2 Files in Scope

The following are the SHA1 hashes of the original reviewed files.

ID	File	SHA-1 Hash
F1B1	sources/fee100bps.move	06403d390dc11b8d219fffa799a43c7ba35ad6f6
PMA	sources/position_manager.move	4818ec48a79ee396bfff23104cd8f692de0c8fc6
F5B	sources/fee500bps.move	bb27c05951a0d6e0ff7717dd9504335983d52918
PFA	sources/pool_factory.move	0ebc8a7f527bed29adf6b15ad56d228d53ed5bd8
FEE	sources/fee.move	56986f1651744aa5c92d59ab0dad924716d11b6b
PFE	sources/pool_fetcher.move	3be5d2c241f492c71972b3be9755b47a96c358d6
POO	sources/pool.move	a2f221916ba4f48b77a464a5f8ffb34381093448
PNF	sources/position_nft.move	e5b54383db42ee9635a9f02923b0e18d28ccee2b
F1B2	sources/fee10000bps.move	25401c155746d735c003b085718663ebb1b7c0e6
F3B	sources/fee3000bps.move	561f33c37fdb13b4ede93e6df533bdd8f5d26df3
SRO	sources/swap_router.move	587c7c11c969d5c91d75b15c454f5f46dd0f0d76

FMU3	sources/lib/full_math_u32.move	49da8c7f42cdb7d5ee47e2fc3abc82fa9c51b7dc
MU2	sources/lib/math_u256.move	88c1460552c88b1a99accdfc202670903d859c74
MU1	sources/lib/math_u128.move	e1dcd290c2149667690c528f8ac462fe485e0524
MLI	sources/lib/math_liquidity.move	d94988b2d7cfe2e5cf026ea89838371378e708c0
FMU6	sources/lib/full_math_u64.move	be9e7b673a23a10242949077c84cf6b888de380b
MBI	sources/lib/math_bit.move	d409aef76da297737d69c4d2771c9dda1aaaaa3d
FMU1	sources/lib/full_math_u128.move	f37cedac47ecdbbd3c11bb684ae525648b8ac19e
STO	sources/lib/string_tools.move	4a6f5cbd89dab7449bad16fd9213cb58e742b805
I32	sources/lib/i32.move	343db86e3cd3d0c18f05b87d4bb63f0e5e223981
I12	sources/lib/i128.move	364951c270ef74ded0522c68af78dfb2ba4e1622
I64	sources/lib/i64.move	90887a84d88c32a1eb4db1768a25436e1017a4ea
MSP	sources/lib/math_sqrt_price.move	4a4d77c1a60cc71f8b32182600f693dffec19ce5
MSW	sources/lib/math_swap.move	82b81b3d13f1cc62978a8229b435fc41edb2493b

MU6	sources/lib/math_u64.move	c488486281bc4d19391ae46ef71219ec45527bf7
MTI	sources/lib/math_tick.move	4d32199cba6c59a6fd5994e5301e3b261a0d381c
RMA	sources/reward_manager.move	1e7e574e016bc61b11647e1365f9b4da5dc067e3
PMA	sources/position_manager.move	2ce72599655eaa646296be7dbb289e5581d3d2cf
PFA	sources/pool_factory.move	a0bf335802ddc72b74664bb19e5aa81b9e56f4e7
POO	sources/pool.move	2ca05e24215268b91a8e2078226aea77ac344d50
SRO	sources/swap_router.move	6b014a73388041799bb921092c0bad5f9b50aae7
STO	sources/lib/string_tools.move	308c957e016540bdd9003d4aeeea8ace3445b9e4
MSW	sources/lib/math_swap.move	8b3ff2152d377080735459e39bf9fe085628e9fc

1.3 Issue Statistic

Item	Count	Fixed	Acknowledged
Total	11	9	2
Informational	1	0	1
Minor	7	7	0
Medium	1	0	1
Major	2	2	0
Critical	0	0	0

1.4 MoveBit Audit Breakdown

MoveBit aims to assess repositories for security-related issues, code quality, and compliance with specifications and best practices. Possible issues our team looked for included (but are not limited to):

- Transaction-ordering dependence
- Timestamp dependence
- Integer overflow/underflow by bit operations
- Number of rounding errors
- Denial of service / logical oversights
- Access control
- Centralization of power
- Business logic contradicting the specification
- Code clones, functionality duplication
- Gas usage
- Arbitrary token minting
- Unchecked CALL Return Values
- The flow of capability
- Witness Type

1.5 Methodology

The security team adopted the **"Testing and Automated Analysis"**, **"Code Review"** and **"Formal Verification"** strategy to perform a complete security test on the code in a way that is closest to the real attack. The main entrance and scope of security testing are stated in the conventions in the "Audit Objective", which can expand to contexts beyond the scope according to the actual testing needs. The main types of this security audit include:

(1) Testing and Automated Analysis

Items to check: state consistency / failure rollback / unit testing / value overflows / parameter verification / unhandled errors / boundary checking / coding specifications.

(2) Code Review

The code scope is illustrated in section 1.2.

(3) Formal Verification(Optional)

Perform formal verification for key functions with the Move Prover.

(4) Audit Process

- Carry out relevant security tests on the testnet or the mainnet;
- If there are any questions during the audit process, communicate with the code owner in time. The code owners should actively cooperate (this might include providing the latest stable source code, relevant deployment scripts or methods, transaction signature scripts, exchange docking schemes, etc.);
- The necessary information during the audit process will be well documented for both the audit team and the code owner in a timely manner.

2 Summary

This report has been commissioned by Turbos to identify any potential issues and vulnerabilities in the source code of the Turbos Clmm smart contract, as well as any contract dependencies that were not part of an officially recognized library. In this audit, we have utilized various techniques, including manual code review and static analysis, to identify potential vulnerabilities and security issues.

During the audit, we identified 11 issues of varying severity, listed below.

ID	Title	Severity	Status
PMA-1	Slippage Protection Error	Major	Fixed
PMA-2	Repeated Assignment	Minor	Fixed
PMA-3	Spelling Error	Minor	Fixed
POO-1	Incorrect `fee_growth_above` Condition	Major	Fixed
POO-2	Unused Error Codes	Minor	Fixed
POO-3	Hardcoded Reward Index in `modify_tick_reward()` and `modify_tick()`	Minor	Fixed
POO-4	Dry Run Mode Accidentally Triggers Events	Informational	Acknowledged
RMA-1	Insufficient Reward Pool May Cause Some Users to Be Unable to Receive Rewards	Medium	Acknowledged
SRO-1	Failure to Check Token Balance in Advance Leads to Redundant Calculations	Minor	Fixed

SRO-2	Spelling Error	Minor	Fixed
STO-1	Position Creation Does Not Take into Account The Case Where The Tick Is 0	Minor	Fixed

3 Participant Process

Here are the relevant actors with their respective abilities within the Turbos Clmm Smart Contract :

Admin

- Admin can set `fee_tier` through set_fee_tier().
- Admin can set `fee_protocol` through set_fee_protocol().
- Admin can update pool `fee_protocol` through update_pool_fee_protocol().
- Admin can collect protocol fee through collect_protocol_fee().
- Admin can collect protocol fee through collect_protocol_fee_with_return_().
- Admin can toggle pool status through toggle_pool_status().
- Admin can update `ntf_name` through update_nft_name().
- Admin can upgrade through upgrade().
- Admin can update `nft_description` through update_nft_description().
- Admin can update `nft_img_url` through update_nft_img_url().
- Admin can migrate position through migrate_position().
- Admin can modify tick through modify_tick().
- Admin can modify reward through modify_reward().
- Admin can init reward through init_reward().
- Admin can reset reward through reset_reward().
- Admin can update reward manager through update_reward_manager().
- Admin can update reward emissions through update_reward_emissions().

User

- User can deploy pool and mint liquidity through deploy_pool_and_mint().
- User can deploy pool through deploy_pool().
- User can compute swap result through compute_swap_result().
- User can mint liquidity through mint().

- User can burn liquidity through burn().
- User can increase liquidity through increase_liquidity().
- User can decrease liquidity through decrease_liquidity().
- User can collect through collect().
- User can collect reward through collect_reward().
- User can burn nft directly through burn_nft_directly().
- User can add reward through add_reward().
- User can remove reward through remove_reward().
- User can swap coin through swap_a_b().
- User can swap coin through swap_b_a().
- User can swap coin through swap_a_b_b_c().
- User can swap coin through swap_a_b_c_b().
- Uuser can swap coin through swap_b_a_b_c().
- Uuser can swap coin through swap_b_a_c_b().
- User can flash swap through flash_swap().
- User can repay the flash swap through repay_flash_swap().

4 Findings

PMA-1 Slippage Protection Error

Severity: Major

Status: Fixed

Code Location:

sources/position_manager.move#535

Descriptions:

The issue is that instead of comparing `amount_b` with `amount_b_min`, the code compares `amount_b_min` with itself (`amount_b_min >= amount_b_min`). This comparison will always evaluate to true, effectively nullifying the slippage protection for token B.

```
assert!(amount_a >= amount_a_min && amount_b_min >= amount_b_min,
EPriceSlippageCheck);
```

Suggestion:

Fix the assertion to properly compare the actual amount received with the minimum amount:

```
assert!(amount_a >= amount_a_min && amount_b >= amount_b_min,
EPriceSlippageCheck);
```

Resolution:

This issue has been fixed. The client has adopted our suggestions.

PMA-2 Repeated Assignment

Severity: Minor

Status: Fixed

Code Location:

sources/position_manager.move#248 249

Descriptions:

When initializing position, first assign `fee_growth_inside_a` and `fee_growth_inside_b` according to the data of the position corresponding to `position_key`.

```
let position_m = Position {
        id: position_id,
        tick_lower_index: tick_lower_index_i32,
        tick_upper_index: tick_upper_index_i32,
        liquidity: liquidity_delta,
        fee_growth_inside_a: pool::get_position_fee_growth_inside_a(pool, position_key),
        fee_growth_inside_b: pool::get_position_fee_growth_inside_b(pool, position_key),
        tokens_owed_a: 0,
        tokens_owed_b: 0,
        reward_infos: vector::empty<PositionRewardInfo>(),
    };
```

Then when it comes to `copy_position()` which calls `pool::get_position_base_info()`:

```
public fun get_position_base_info<CoinTypeA, CoinTypeB, FeeType>(
    pool: &Pool<CoinTypeA, CoinTypeB, FeeType>,
    key: String
): (u128, u128, u128, u64, u64, &vector<PositionRewardInfo>) {
    let position = get_position_by_key(pool, key);
    (
        position.liquidity,
        position.fee_growth_inside_a,
        position.fee_growth_inside_b,
```

```
        position.tokens_owed_a,
        position.tokens_owed_b,
        &position.reward_infos
    )
}
```

As you can see,it get a position corresponding to `key` which is position_key essentially. So `fee_growth_inside_a` and `fee_growth_inside_b` are assigned the same value twice.

Suggestion:

It is recommended to set the first value to 0 to reduce redundant operations.

```
let position_m = Position {
        id: position_id,
        tick_lower_index: tick_lower_index_i32,
        tick_upper_index: tick_upper_index_i32,
        liquidity: liquidity_delta,
        fee_growth_inside_a: 0
        fee_growth_inside_b: 0
        tokens_owed_a: 0,
        tokens_owed_b: 0,
        reward_infos: vector::empty<PositionRewardInfo>(),
    };
```

Resolution:

This issue has been fixed. The client has adopted our suggestions.

PMA-3 Spelling Error

Severity: Minor

Status: Fixed

Code Location:

sources/position_manager.move#667

Descriptions:

There are some cases where variable names are misspelled, for example `reawrd_info` should be `reward_info`:

```
public(friend) fun modify_tick<CoinTypeA, CoinTypeB, FeeType>(
    pool: &mut Pool<CoinTypeA, CoinTypeB, FeeType>,
    tick_index: I32,
) {
    if (i32::lte(tick_index, pool.tick_current_index)) {
        let reward_infos = &pool.reward_infos;
        let reawrd_info = vector::borrow(reward_infos, 0);
        modify_tick_reward_outside(pool, tick_index, 0, reawrd_info.growth_global);
    } else {
        modify_tick_reward_outside(pool, tick_index, 0, 0);
    };
}
```

Suggestion:

It is recommended to fix this spelling error.

Resolution:

This issue has been fixed. The client has adopted our suggestions.

POO-1 Incorrect `fee_growth_above` Condition

Severity: Major

Status: Fixed

Code Location:

sources/pool.move#1553

Descriptions:

The condition if (`!tick_lower.initialized`) is used to determine if the fee growth above should be set to zero. However, this condition should be checking `tick_upper.initialized` instead. The current logic incorrectly assumes that the lower tick's initialization status is relevant for calculating the fee growth above the current tick range.

```
if (!tick_lower.initialized) {
        fee_growth_above_a = 0;
        fee_growth_above_b = 0;
//...
```

Suggestion:

Fix the condition to check the upper tick's initialization status:

```
if (!tick_upper.initialized) {
   fee_growth_above_a = 0;
   fee_growth_above_b = 0;
//...
```

Resolution:

This issue has been fixed. The client has adopted our suggestions.

POO-2 Unused Error Codes

Severity: Minor

Status: Fixed

Code Location:

sources/pool.move#48,49;

sources/position_manager.move#26,27,28,29;

sources/swap_router.move#14,16

Descriptions:

Unused error codes were found in the smart contract, likely due to changes in the code logic without timely cleanup. This can reduce code readability, mislead reviewers, and increase maintenance costs. For example,in `pool.move` #48,49.

```
const ESwapLessThanMinSqrtPrice: u64 = 9;
const ESwapGatherThanMaxSqrtPrice: u64 = 10;
```

Suggestion:

It is recommended to remove unused error codes, ensure the integrity of the logic, and update related documentation or comments to improve code simplicity and maintainability.

Resolution:

This issue has been fixed. The client has adopted our suggestions.

POO-3 Hardcoded Reward Index in `modify_tick_reward()` and `modify_tick()`

Severity: Minor

Status: Fixed

Code Location:

sources/pool.move#2145

Descriptions:

```
public(friend) fun modify_tick<CoinTypeA, CoinTypeB, FeeType>(
    pool: &mut Pool<CoinTypeA, CoinTypeB, FeeType>,
    tick_index: I32,
) {
    if (i32::lte(tick_index, pool.tick_current_index)) {
        let reward_infos = &pool.reward_infos;
        let reawrd_info = vector::borrow(reward_infos, 0);  // Always uses index 0
        modify_tick_reward_outside(pool, tick_index, 0, reawrd_info.growth_global);
    } else {
        modify_tick_reward_outside(pool, tick_index, 0, 0);  // Always uses index 0
    };
}
```

`modify_tick_reward()` and `modify_tick()` in the pool contract incorrectly handle reward indices by hardcoding them to 0, leading to broken reward accounting for all reward tokens beyond the first one.

Suggestion:

It is recommended to add another input parameter `reward_index` to specify which reward token to modify.

Resolution:

This function was originally scheduled for removal and is no longer used. It has now been removed.

POO-4 Dry Run Mode Accidentally Triggers Events

Severity: Informational

Status: Acknowledged

Code Location:

sources/pool.move#681

Descriptions:

When using dry run mode for swapping, events should not be released, which may cause misunderstandings in on-chain monitoring or front-end data capture errors.

```
if (!dry_run) {
        //...
    };
    state.amount_a = amount_a;
    state.amount_b = amount_b;

    event::emit(SwapEvent {
        pool: object::id(pool),
        recipient: recipient,
        amount_a: (state.amount_a as u64),
        amount_b: (state.amount_b as u64),
        liquidity: state.liquidity,
        tick_current_index: state.tick_current_index,
        tick_pre_index: tick_pre_index,
        sqrt_price: state.sqrt_price,
        protocol_fee: (state.protocol_fee as u64),
        fee_amount: (state.fee_amount as u64),
        a_to_b: a_to_b,
        is_exact_in: amount_specified_is_input,
    });
```

Suggestion:

It is recommended not to release events in dryrun mode.

RMA-1 Insufficient Reward Pool May Cause Some Users to Be Unable to Receive Rewards

Severity: Medium

Status: Acknowledged

Code Location:

sources/reward_manager.move

Descriptions:

The rewards a user receives are only related to the time and the set reward release speed. Before receiving the rewards, the user does not know whether the reward vault has sufficient funds, and the release speed and time are not related to the reward amount, which means that the total amount of reward release may be greater than the actual reward amount, which will cause the user's loss of interests.

For example, Alice and Bob both received 100 reward tokens, but there are only 80 reward tokens in the reward vault. For Alice and Bob, being able to receive this reward depends only on who takes the first action to call the function, which is unfair to the party who did not receive it.

The reason for this is that the total amount of released rewards may not match the actual reward amount.

Suggestion:

It is recommended that the number of released rewards should not exceed the actual number of rewards, or other measures should be taken to ensure that there will be no reward run.

Resolution:

Developer Response: Off-chain tasks check reward balance and emissions; if the balance is insufficient, emissions will stop.

SRO-1 Failure to Check Token Balance in Advance Leads to Redundant Calculations

Severity: Minor

Status: Fixed

Code Location:

sources/swap_router.move#60,137

Descriptions:

In the function `swap_a_b_with_return_()` and `swap_b_a_with_return_()`, the amount of coin amount check is performed after extensive computations on the input parameter. In `swap_router.move` #137:

```
let (amount_a, amount_b) = pool::swap(
            pool,
            recipient,
            false,
            (amount as u128),
            is_exact_in,
            sqrt_price_limit,
            clock,
            ctx
        );
        let amount_a_64 = (amount_a as u64);
        let amount_b_64 = (amount_b as u64);
        check_amount_threshold(is_exact_in, false, amount_a_64, amount_b_64,
amount_threshold);

        pool::swap_coin_b_a_with_return_(
            pool,
            pool::merge_coin(coins_b),
            amount_b_64,
```

```
        amount_a_64,
        ctx
    )
```

For example,when coin B is less than the specified amount for swapping,the function would not end until it completes calculation of swapping.

Suggestion:

To optimize performance and avoid redundant calculations, it is recommended to move the coin amount check to the beginning of the function.

Resolution:

This issue has been fixed. The client has adopted our suggestions.

SRO-2 Spelling Error

Severity: Minor

Status: Fixed

Code Location:

sources/swap_router.move#15

Descriptions:

There is a typo in the error code here

```
const ETransactionToOld: u64 = 2;
```

`ETransactionToOld` should be spelled to `ETransactionTooOld`.

Suggestion:

It is recommended to fix this typo.

Resolution:

This issue has been fixed. The client has adopted our suggestions.

STO-1 Position Creation Does Not Take into Account The Case Where The Tick Is 0

Severity: Minor

Status: Fixed

Code Location:

sources/lib/string_tools.move#13

Descriptions:

According to the function annotation, the function is intended to determine the unique position through the `(owner, tickLower, tickUpper)` triple. In actual implementation, the function marks the sign according to the positive or negative of the tick.

```
let tick_lower_index_str = u64_to_string((tick_lower_index as u64));
let tick_lower_index_is_neg_str = if(tick_lower_index_is_neg) string::utf8(b"-") else string::utf8(b"+");
let tick_upper_index_str = u64_to_string((tick_upper_index as u64));
let tick_upper_index_is_neg_str = if(tick_upper_index_is_neg) string::utf8(b"-") else string::utf8(b"+");
```

Since the positive and negative values of 0 are both 0, this allows users to create multiple position keys while means the same ticks of position, for example, positions -0 to 10 ticks and +0 to 10 ticks. This is an operation outside of position design and may have potential attack risks.

Suggestion:

It is recommended to consider the special case of 0 when creating position keys.

Resolution:

This issue has been fixed. The client has adopted our suggestions.

Appendix 1

Issue Level

- **Informational** issues are often recommendations to improve the style of the code or to optimize code that does not affect the overall functionality.
- **Minor** issues are general suggestions relevant to best practices and readability. They don't post any direct risk. Developers are encouraged to fix them.
- **Medium** issues are non-exploitable problems and not security vulnerabilities. They should be fixed unless there is a specific reason not to.
- **Major** issues are security vulnerabilities. They put a portion of users' sensitive information at risk, and often are not directly exploitable. All major issues should be fixed.
- **Critical** issues are directly exploitable security vulnerabilities. They put users' sensitive information at risk. All critical issues should be fixed.

Issue Status

- **Fixed:** The issue has been resolved.
- **Partially Fixed:** The issue has been partially resolved.
- **Acknowledged:** The issue has been acknowledged by the code owner, and the code owner confirms it's as designed, and decides to keep it.

Appendix 2

Disclaimer

This report is based on the scope of materials and documents provided, with a limited review at the time provided. Results may not be complete and do not include all vulnerabilities. The review and this report are provided on an as-is, where-is, and as-available basis. You agree that your access and/or use, including but not limited to any associated services, products, protocols, platforms, content, and materials, will be at your own risk. A report does not imply an endorsement of any particular project or team, nor does it guarantee its security. These reports should not be relied upon in any way by any third party, including for the purpose of making any decision to buy or sell products, services, or any other assets. TO THE FULLEST EXTENT PERMITTED BY LAW, WE DISCLAIM ALL WARRANTIES, EXPRESS OR IMPLIED, IN CONNECTION WITH THIS REPORT, ITS CONTENT, RELATED SERVICES AND PRODUCTS, AND YOUR USE, INCLUDING BUT NOT LIMITED TO THE IMPLIED WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE, NOT INFRINGEMENT.